

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS1IP: Imperative Programming

Week 12: Security and Source Code Licensing

Learning Outcomes

- What is Software Security and Adversarial input?
- What is source source licensing?

Introduction to Software Security

What is Software Security?

- It involves the protection of software applications from various types of threats and vulnerabilities that could compromise their:

Why is Software Security Important?

- **Protect Sensitive Data:** Safeguard personal and financial information.
- **Maintain Integrity:** Ensure software performs as intended without tampering.
- **Prevent Financial Loss:** Avoid costly breaches and legal consequences.
- **Build Trust:** Enhance user confidence in the application.

Common Software Security Threats

- Malware: Malicious software designed to harm or exploit.
 - Example: Ransomware encrypts data and demands payment for decryption.
- Phishing: Attempts to trick users into divulging sensitive information.
 - Example: Fake emails that appear to be from legitimate sources asking for passwords.
- SQL Injection: It is an adversarial input to exploit vulnerabilities in database queries.
 - Example: Injecting malicious SQL code to manipulate the database

Adversarial input attacks

In 2005, Mizuho Securities, Japanese investment company, suffered a massive financial loss due a typing error. A broker intended to sell on share of J-Com Co. at ¥610,000 but ended up mistakenly placed an order to sell 610,000 shares at ¥1 each. This typo resulted in loss of ¥27 billion (approx. USD\$225M) for the firm.

i Who is the fault? the broker or the system broker



What are Adversarial Inputs?

- Maliciously crafted inputs intended to exploit vulnerabilities in a program.
 - .. *even or some bad input such in the Mizuho case.*

Why it is important:

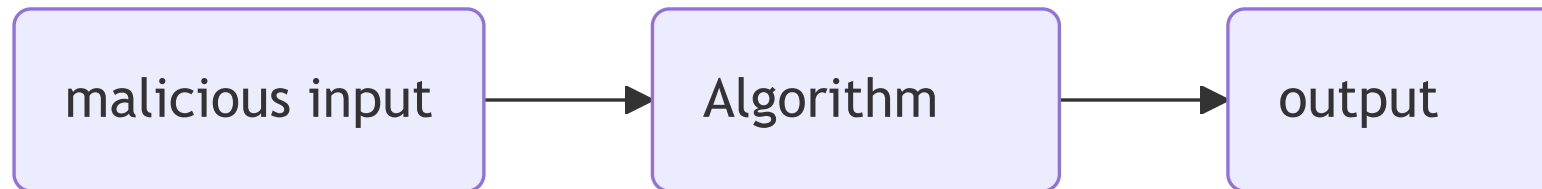
- Understanding how adversarial inputs work helps in designing more secure and robust search algorithms.

What are Adversarial Inputs?

Expected input



Adversarial Inputs



Types of Adversarial Input attacks

- Algorithmic Complexity Attacks
- Buffer Overflow Attacks
- Injection Attacks

Algorithmic Complexity Attacks

- Target: Efficiency of the search algorithm.
- Example: Providing input that causes excessive recursion or deep tree traversal, slowing down the search.

Example

```
boolean search(List<Integer> list, int target) {  
    return list.contains(target);  
}  
// Adversarial Input Example: A linked list with 1 million zeros followed by a 1  
List<Integer> adversarialInput = new LinkedList<>(Collections.nCopies(1_000_000, 0));  
adversarialInput.add(1);  
search(adversarialInput, 1); // Inefficient for linked lists due to linear traversal
```

This input forces the algorithm to traverse a large linked list inefficiently before finding the target, exploiting the complexity of `contains()` for linked lists

Buffer Overflow Attacks

- Target: Memory management during search operations.
- Example: Overflowing buffers to alter the control flow of a program.

Example

```
void processInput(String adversarialInput, int amountToCopy) {  
    byte[] buffer = new byte[16];  
    for (int i = 0; i < amountToCopy; i++) {  
        buffer[i] = (byte) adversarialInput.charAt(i); // Potential ArrayIndexOutOfBoundsException  
    }  
  
    System.out.println("Buffer content: ");  
    for (byte b : buffer) {  
        System.out.print((char) b);  
    }  
    System.out.println();  
}  
  
String adversarialInput = "AAAAAAAAAAAAAAAAAAAAAAAAAAAA";  
int amountToCopy = 24; // Intentionally causes a buffer overflow  
processInput(adversarialInput, amountToCopy);
```

Buffer Overflow Attacks

"Heartbleed" incident

- A critical security vulnerability in the OpenSSL cryptographic library.
- Affected versions: 1.0.1 to 1.0.1f (2012-2014).
- The server would return data from memory beyond the intended buffer.
- Trust erosion in websites services
 - Millions of Websites Affected, such as Yahoo, GitHub, Flickr, and Tumblr were compromised.
 - Data Exposure: Passwords, Private encryption keys, Usernames and emails.



SQL Injection Attacks

- Target: Search queries or input fields.
- Example: SQL injection in search forms, where adversarial inputs manipulate database queries.

Example

```
SELECT * FROM users WHERE username = 'admin' AND password = '' OR '1'='1';
```

Explanation: The input manipulates the SQL query to bypass authentication by always evaluating to true.

How to prevent attacks?

There are full training for security analysts

i But why how prevent such attacks as initial programmers?

- Think about careful about your code.
- Thread the user an silly
- Use **Defensive Programming...**

program design intended to develop programs that are capable of detecting potential security abnormalities and make predetermined responses

Defensive Programming Example: Preventing Null and Empty Inputs

```
boolean search(int[] arr, int target) {  
    if (arr == null || arr.length == 0) {  
        throw new IllegalArgumentException("Input array is null or empty");  
    }  
    for (int i : arr) {  
        if (i == target) {  
            return true;  
        }  
    }  
    return false;  
}
```

```
search(new int[]{1, 2, 3}, 2); // Returns true
```

Defensive Programming Example: Sanitizing User Input

```
String sanitizeInput(String input) {  
    return input.replaceAll("[^a-zA-Z0-9]", "");  
}  
  
String userInput = "<script>alert('Hack!')</script>";  
System.out.println(sanitizeInput(userInput)); // Outputs: alertHack
```

Defensive Programming Example: Handling Division by Zero

In defensive programming, you should always check for potential division by zero to prevent runtime exceptions.

```
double safeDivide(double numerator, double denominator) {  
    if (denominator == 0) {  
        throw new IllegalArgumentException("Denominator cannot be zero");  
    }  
    return numerator / denominator;  
}
```

```
System.out.println(safeDivide(10, 2)); // Outputs: 5.0  
System.out.println(safeDivide(10, 0)); // Throws IllegalArgumentException
```

This code checks if the denominator is zero before performing the division, preventing a `ArithmeticException` and ensuring predictable behavior.

Defensive Programming Example: Checking Array Index Bounds

When accessing an array, ensure the index is within valid bounds to avoid `ArrayIndexOutOfBoundsException`.

```
int getElement(int[] arr, int index) {  
    if (arr == null) {  
        throw new IllegalArgumentException("Array cannot be null");  
    }  
    if (index < 0 || index >= arr.length) {  
        throw new IndexOutOfBoundsException("Index out of bounds");  
    }  
    return arr[index];  
}  
  
int[] numbers = {1, 2, 3, 4, 5};  
System.out.println(getElement(numbers, 2)); // Outputs: 3  
System.out.println(getElement(numbers, 10)); // Throws IndexOutOfBoundsException
```

in-class quiz 2

Week 11: Which type of attack exploits memory management vulnerabilities?

- SQL Injection
- Phishing
- Buffer Overflow
- Malware

Open Source licensing

What is Software Licensing?

Software licensing refers to the legal terms under which software **can be used, modified, and distributed**.

- Defines the **rights and restrictions** for both developers and users.
- Defines how others can use, modify, and distribute your code.
- **Protects the creators rights.**

Types of Software Licensing

1. Proprietary Licenses:

- Software is owned by an entity.
- Users cannot modify or redistribute the source code.
- Examples: Microsoft Windows, Adobe Photoshop, Skype.

2. Open-Source Licenses:

- Source code is publicly available for modification and redistribution.
- Examples: Linux, Apache, MySQL.

Open-Source Licensing

Why Licensing Matters?

- **Reusability:** Reuse and modify existing open-source libraries.
- **Collaboration:** Contribute to open-source projects or use them in your work.
Example: share a code for a common task
- **Compliance:** Ensure you're following legal guidelines when using open-source code. Example: reference software for a standard

Who invented the Open Source?

- The term "Open Source" was coined by Eric S. Raymond and Bruce Perens
- But Richard Stallman is known for having the most significant contribution with the concept of "Free Software"

Richard Stallman's Contributions to Open-source

- Founder of the Free Software Movement:
 - In 1983, Stallman launched the GNU Project to create a free Unix-like operating system.
- GNU General Public License (GPL):
 - Introduced in 1989, ensuring software remains free to use, modify, and distribute.
- Philosophy:
 - Focuses on user freedom over software development practices.



Famous examples of open-source software

- GNU GCC (C language compiler)
- Linux
- Android
- OpenSSI

i But that happen in the corporative world as well



Open-source in corporations

snowflake-connector-python
dependency on open-source
oscrypto

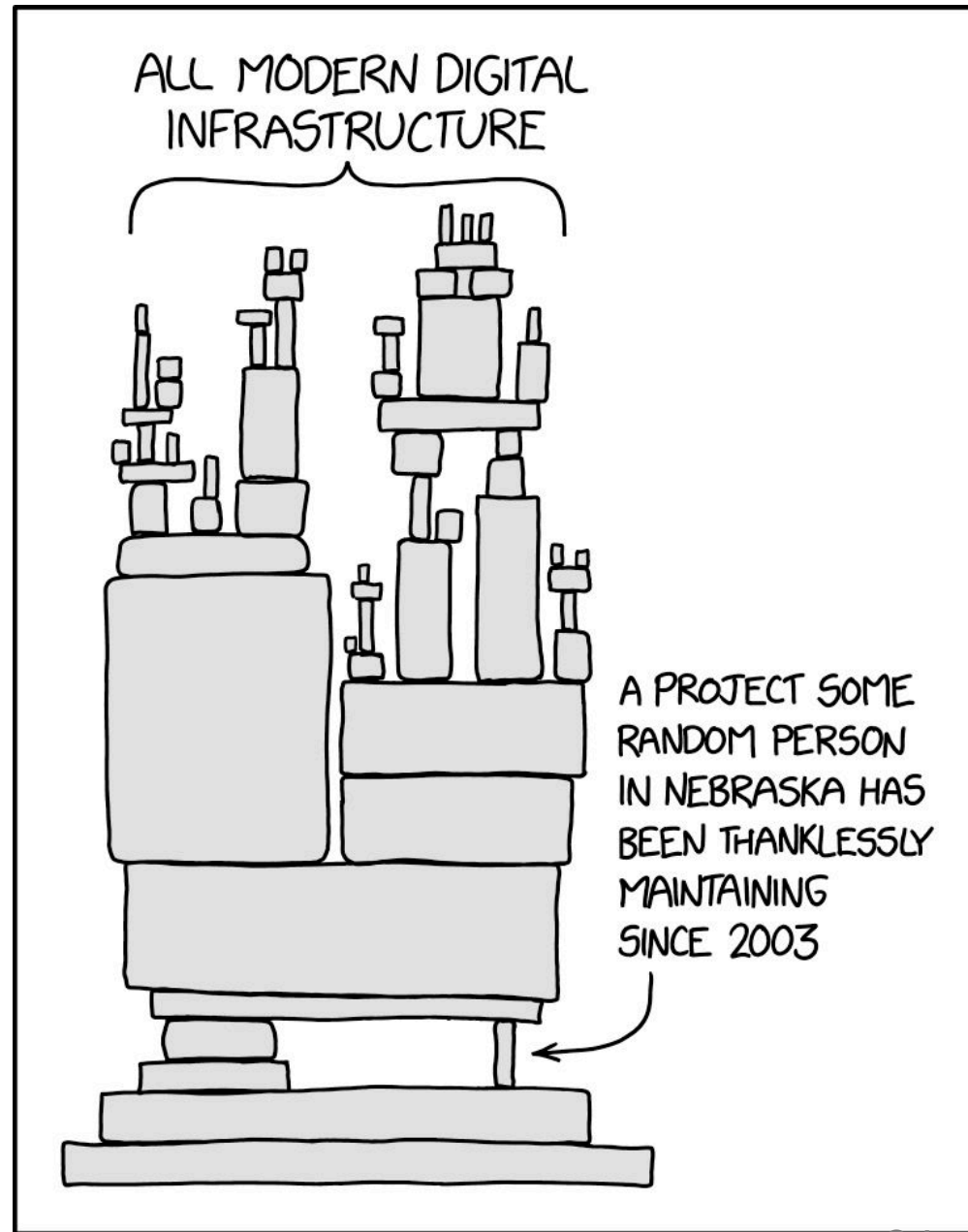


Fernando Brito • 1st
Data Platform Engineer | Consultant | Speaker | ❄️ Snowflake Data Superhero
1yr • Edited • 🔄

🌟 Yesterday there was an incident affecting some of the data pipeline infrastructure that my team owns. It was a great example of the often-forgotten reality shown in the famous "xkcd" comic below [1].

"oscrypto" is maintained by one individual in his free time. The last release was over a year ago, and the one before, more than 3 years ago. A fix for the regular expression has just been merged into their main branch, but the maintainer did not have time yet to make a new version release including it [7]. Unfortunately, there have been some comments on his project criticizing him for taking too long to take action [8], but I don't think it's his fault that his free time project is now used in critical infrastructure around the world.

🔴 Until there's a new release of "oscrypto", this issue is just like a time bomb. As more and more Docker images are rebuilt and have OpenSSL version 3.0.10 or 3.0.11 installed, more and more systems will crash. There's an OpenSSL 3.1.x release track that is not affected (they are in 3.1.3), so if you are lucky, your Docker image will already use it. Anyway, watch out for your data applications and be safe out there.



Microsoft:

- Acquired **GitHub** (2018).
- Major contributions to **Linux**, **VSCode**, and more.

IBM:

- Acquired **Red Hat** (2019) for \$34 billion.
- Significant contributions to **Kubernetes** and **Apache** projects.

Google:

- Founded and maintains **Kubernetes** and **TensorFlow**.
- Heavy contributions to **Linux Kernel** and **Go** language.

Meta (Facebook):

- Created projects like **React**, **PyTorch**, and **GraphOL**



Types of open-source licenses

There are many open-source licenses

- Each open-source group can create yours
- Examples are Apache, MIT, GNU, Mozilla etc

But how categorize them

You can see them as:

- Copyleft Licenses (Stallman "Free Software")
- Permissive Licenses

Copyleft Licenses

GNU General Public License (GPL)

- Permissions:
 - Free to use, modify, and distribute.
 - Any derivative work must also be open-sourced under GPL.
- Conditions:
 - Source code must be made available.
 - Modifications must also be licensed under GPL.

Lesser GPL (LGPL)

- Permissions:
 - Allows linking to proprietary code under certain conditions.
- Conditions:

Permissive Licenses

MIT License

- Permissions:
 - Free to use, copy, modify, and distribute.
 - Can be used in proprietary software.
- Conditions:
 - Must include the original license and copyright notice.

Apache License 2.0

- Permissions:
 - Similar to MIT, with an added patent grant.
- Conditions:
 - Must include a copy of the license.

How Choosing a License for Your Project

Collaboration

- Do you want others to contribute to your project?

Commercial Use

- Are you okay with your code being used in proprietary software?

Copyleft vs. Permissive

- Do you want to ensure that all derivatives remain open-source?

Tools

- choosealicense.com website helps in selecting a license.
- GitHub License Templates: Provides easy-to-use templates for common licenses.

How to Apply a License to Your Code

- Include a License File:
 - Add a `LICENSE` or `COPYING` file to the root of your project.
- Add License Information to Source Files:
 - Include a comment at the top of each source file indicating the license.

Example: MIT License Header

```
/*
 * MIT License
 * Copyright (c) 2024 Your Name
 *
 * Permission is hereby granted, free of charge, to any person obtaining a copy
 * of this software and associated documentation files (the "Software"), to deal
 * in the Software without restriction, including without limitation the rights
 * to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
 * copies of the Software, and to permit persons to whom the Software is
 * furnished to do so, subject to the following conditions:
 *
 * The above copyright notice and this permission notice shall be included in all
 * copies or substantial portions of the Software.
 */
```

in-class quiz 3

Week 11: Which of the following is a characteristic of permissive open-source licenses?

- Requires derivative works to be released under the same license.
- Allows proprietary use and distribution with minimal restrictions.
- Requires all modifications to be made public.
- Prohibits commercial use of the software.

Best Practices

- Best Practices:
 - Always include a license in your projects.
 - Understand the licenses of the libraries you use.
 - Choose a license that aligns with your project goals.

Pitfalls to Avoid

- Ignoring License Requirements:
 - Ensure you comply with the terms of the licenses you use.
- Mixing Incompatible Licenses:
 - Be aware of conflicts when combining code from different sources.

Review Learning Outcomes

Review Learning Outcomes

Is Imperative Programming the only way to program?

- There are other programming paradigms such as Declarative Programming and Functional programming

What is Software Security and Adversarial input?

- Software Security is essential to main software works as expected.
- Adversarial inputs are attacks of programs that receives input from user.

What is Open source licensing?

- Open source licensing is a essential piece of Computer Science field.
 - For learning, standards, collaboration, etc.

DEPARTMENT OF COMPUTER SCIENCE



Thank you! 😊

Do you have any questions 🙋 ?